EXPERIMENT -1

Comparative study of low-code development versus traditional application development approaches

Aim

To compare the low-code approach to building applications with the traditional (hand-coded) approach, using the Job Application System (the "Recruitment & Job Application Management" application) as a practical example, and to state when each approach is the better choice.

Procedure

1. **Choose the example system.** The Job Application System comes from Assignment 05. HR receives applications by email, walk-ins and referrals in inconsistent formats. The goal is one online form that captures name, email, address, phone, LinkedIn link, cover letter, years of experience, joining date, interview time, department, employment type, key skills and a consent checkbox, followed by a report for HR and an acknowledgement email.
2. **Define the two approaches** in simple words (see Definitions below).
3. **Select comparison criteria**: skills needed, development speed, cost, flexibility, maintenance, scalability, security and governance, integration, and vendor dependence.
4. **List the work needed to build the Job Application System** in the traditional way and in the low-code way (see Work breakdown below).
5. **Fill the comparison table** for every criterion.
6. **Analyse the table**, note the limitations of each approach, and write the conclusions.

Definitions

Traditional application development means writing the application by hand in a programming language. The team designs the database, writes the server (back-end) code, builds the screens (front-end), sets up hosting and security, tests, and then deploys. Every part is under the developers' full control, but every part also has to be built and maintained.

Low-code development means building the application mostly by dragging and configuring ready-made components on a visual platform such as Zoho Creator, and writing small pieces of script only where needed. The platform supplies the database, hosting, security basics, forms, reports and workflow engine.

Work breakdown for the Job Application System

Task	Traditional approach	Low-code approach (Zoho Creator)
Store applicant data	Design tables and columns, choose and install a database, write queries.	Create a form; the platform stores each submission as a record.
Input screens	Write HTML/CSS/JavaScript for 13 inputs (text, email, address, phone, number, date, time, drop-down, radio, multi-select, checkbox).	Drag the matching field types onto the form.
Validation	Write and test code for mandatory fields, email format, 0-40 experience range, no past joining date, consent required.	Switch on field properties such as mandatory, range and date limits (as listed in Assignment 05).
List and filter applicants	Write an API and a table page with filters and paging.	Create a report from the form and use its built-in filters.
Acknowledgement email	Set up a mail server or service, write the sending code and handle failures.	Configure a workflow that sends an email on form submission.
Public link for candidates	Buy hosting, configure a server, domain and HTTPS, deploy.	Publish/share the form using the platform's link feature.
Sign-in and access control	Build login, roles and permissions.	Use the platform's user and role management.

Output

Comparison of the two approaches

Criterion	Traditional development	Low-code development
Skills needed	Professional programmers (front-end, back-end, database, DevOps).	Business analysts and "citizen developers" can build basic apps; developers are needed for advanced logic.
Speed of first version	Slower, because everything is built from the ground up.	Usually faster, because forms, reports and workflows are ready-made.
Cost pattern	Higher effort and staff cost up front and for upkeep; no platform licence.	Less development effort, but a subscription or licence fee to the platform.
Flexibility	Almost unlimited; any design, library or algorithm can be used.	Limited to what the platform offers; extra scripting or pro-code extensions are needed beyond that.
Maintenance	The team maintains servers, updates, patches and code.	The vendor maintains the platform; the team maintains only its own configuration.

Scalability and performance control	Full control over architecture and tuning.	Handled by the platform; less control, but less work.
Security and governance	Entirely the team's responsibility.	Platform provides built-in security and admin controls; the team still must configure them correctly.
Integration	Any system can be connected if code is written for it.	Ready-made connectors for the vendor's own services; others through APIs.
Vendor dependence	Low; the code belongs to the team.	Higher; the application lives on the vendor's platform and plan.
Best suited for	Large, complex or highly customised systems.	Business workflows, forms, internal tools and quick prototypes such as the Job Application System.

Findings and conclusions

1. For a form-centred business process like the Job Application System, the low-code approach removes most of the repetitive work (database set-up, hosting, validation code, list screens and email sending), so the student or HR team can focus on the process itself.
2. The traditional approach gives complete freedom, but for this small use case that freedom would cost much more development effort than the use case needs.
3. Low-code is not free of limits: the application depends on the platform, its plans and its available features. When requirements outgrow the platform, pro-code extensions or a traditional build become necessary.
4. The two approaches are not opponents. Many organisations use low-code for quick business applications and traditional coding for complex, core systems.

Result

The comparative study of low-code and traditional application development was completed as a conceptual exercise, using the Job Application System as the example. Low-code was found to be better suited to quick, form-based business applications, while traditional development is better for highly customised or complex systems. Nothing was built or measured in Zoho Creator for this experiment.